

Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский физико-технический институт (государственный университет)»
Физтех-школа аэрокосмических технологий
Кафедра логистических систем и технологий

Направление подготовки: 27.03.03 Системный анализ и управление

Направленность (профиль) подготовки: Системный анализ и управление в технических,
экономических и социальных системах

**ИССЛЕДОВАНИЕ И РАЗРАБОТКА МЕТОДОВ ЛОКАЛЬНОГО ОПРЕДЕЛЕНИЯ
ЭМОЦИОНАЛЬНОЙ ОКРАСКИ ТЕКСТОВЫХ СООБЩЕНИЙ В МОБИЛЬНЫХ
СИСТЕМАХ**

(бакалаврская работа)

Студент:

Саманюк Эвелина Эдуардовна

(подпись студента)

Научный руководитель:

Булычев Александр Викторович

(подпись научного руководителя)

Москва 2026

Аннотация

В работе исследуется задача локального определения эмоциональной окраски коротких русскоязычных сообщений с возможностью последующего применения в мобильной клавиатуре. Актуальность работы связана с противоречием между качеством интеллектуальных функций клавиатуры и требованиями приватности пользовательского ввода.

Проведен обзор подходов к анализу эмоций и тональности текста, включая словарные методы, классические модели машинного обучения, subword-модели и BERT-подобные модели. Для экспериментальной проверки использован корпус RuGoEmotions, приведенный к шести классам: тревожность, злость, грусть, забота, радость и нейтральное состояние. Сформированы обучающая, валидационная и тестовая выборки общим объемом 7200 сообщений.

Сравнены четыре кандидата: словарный baseline, TF-IDF + Logistic Regression, fastText supervised и RuBERT-tiny2. Выбор итоговой модели выполнен не как автоматическая максимизация одной сводной формулы, а как сравнительный анализ качества классификации и ресурсных ограничений. Словарный baseline использован как нижняя граница, fastText – как быстрый subword-кандидат, RuBERT-tiny2 – как наиболее качественная, но более тяжелая модель, а TF-IDF + Logistic Regression выбрана для прототипа как лучший по макро F1 вариант среди легких локально запускаемых моделей. На ее основе реализован исследовательский Python-прототип локального модуля, который классифицирует сообщения, возвращает оценки уверенности по эмоциональным классам и строит эмоциональный календарь по диалогам без необходимости хранения исходных сообщений после агрегации.

Практический результат работы состоит в реализации прототипа локального модуля для анализа динамики эмоциональной окраски отправленных сообщений и демонстрации его работы на русскоязычных диалогах.

Оглавление

Аннотация	2
Обозначения и сокращения	4
Введение	5
1 Анализ предметной области и обзор литературы	8
1.1 Актуальность и постановка проблемы	8
1.2 Обзор существующих подходов	9
1.3 Обоснование выбора моделей для сравнения	14
1.4 Выводы по главе	15
2 Постановка задачи и выбор методов решения	16
2.1 Формальная постановка задачи	16
2.2 Математический аппарат и обоснование выбранных методов	18
2.3 Выводы по главе	21
3 Реализация и экспериментальная проверка	23
3.1 Описание реализации	23
3.2 Эксперименты и результаты	24
3.3 Построение эмоционального календаря по диалогам	29
3.4 Анализ результатов	31
3.5 Ограничения, этика и приватность	33
3.6 Выводы по главе	34
Заключение	35
Список использованных источников	37
Приложение	41

Обозначения и сокращения

ВКР выпускная квалификационная работа.

NLP обработка естественного языка.

TF-IDF статистическая мера важности термина в документе относительно корпуса.

BERT трансформерная языковая модель для получения контекстных представлений текста.

CPU центральный процессор.

RAM оперативная память.

RSS объем физической памяти, занимаемой процессом в операционной системе.

CSV табличный текстовый формат хранения данных.

F1 гармоническое среднее precision и recall.

НИР научно-исследовательская работа.

Введение

Актуальность темы. Мобильная клавиатура является одним из наиболее часто используемых интерфейсов взаимодействия пользователя со смартфоном. Через нее вводятся поисковые запросы, личные сообщения, рабочая переписка, данные учетных записей и иная информация, которая может иметь конфиденциальный характер. Современные клавиатуры, распространяемые крупными технологическими компаниями, как правило, используют развитые механизмы предиктивного ввода, автокоррекции и персонализации. Высокое качество этих функций часто достигается за счет сбора статистики пользовательского ввода, синхронизации словарей и обработки данных на удаленных серверах.

Такой подход создает противоречие между удобством ввода и требованиями приватности. С одной стороны, пользователь ожидает от клавиатуры точного предсказания слов, адаптации к индивидуальному словарю и поддержки контекстных сценариев. С другой стороны, передача фрагментов пользовательского текста на серверы сторонних организаций повышает риски утечки персональных данных, несанкционированного анализа коммуникации и непрозрачного профилирования. В связи с этим возрастает интерес к офлайн-клавиатурам и локальным алгоритмам обработки текста, которые не требуют отправки пользовательских сообщений за пределы устройства.

Однако полностью локальные клавиатуры часто оказываются менее конкурентоспособными по сравнению с облачными решениями: они хуже учитывают индивидуальные языковые привычки пользователя, ограничены ресурсами мобильного устройства и имеют менее развитые модели анализа контекста. Одним из перспективных направлений повышения их функциональности является разработка локальных интеллектуальных модулей, способных извлекать из коротких текстовых сообщений дополнительные признаки без нарушения приватности. В частности, локальное определение эмоциональной окраски сообщений может быть использовано для построения динамики эмоциональной окраски отправленных сообщений, например в форме эмоционального календаря, при сохранении всех вычислений на устройстве.

Таким образом, актуальность работы определяется необходимостью поиска баланса между качеством интеллектуальных функций мобильной клавиатуры и защитой пользовательских данных, а также потребностью в исследовании моделей обработки коротких текстов, пригодных для локального запуска на мобильных устройствах.

Объект исследования. Объектом исследования являются методы локальной обработки коротких текстовых сообщений в мобильных пользовательских интерфейсах, в частности в системах текстового ввода на смартфонах.

Предмет исследования. Предметом исследования являются алгоритмы и модели определения эмоциональной окраски коротких пользовательских сообщений, а также воз-

возможность их применения в составе офлайн-модуля мобильной клавиатуры с учетом ограничений по вычислительным ресурсам, задержке обработки и приватности данных.

Цель работы. Целью работы является исследование и реализация подхода к локальному определению эмоциональной окраски коротких текстовых сообщений с возможностью последующего применения в мобильной клавиатуре.

Задачи работы.

1. Провести сравнительный анализ подходов и доступных моделей для оценки эмоциональной окраски коротких текстовых сообщений.
2. Выделить критерии выбора модели для мобильного применения, включая качество классификации, размер модели, скорость работы и возможность локального запуска.
3. Выбрать оптимальную модель или группу моделей для реализации офлайн-модуля анализа эмоциональной окраски.
4. Разработать прототип локального программного модуля, имитирующий ключевой сценарий обработки сообщений в мобильной клавиатуре и позволяющий оценивать динамику эмоциональной окраски отправленных сообщений во времени.
5. Провести оценку результатов работы модуля и сформулировать план дальнейшего развития продукта.

Методы исследования. В работе используются методы анализа научно-технической литературы и существующих программных решений, сравнительный анализ моделей обработки естественного языка, экспериментальная оценка качества классификации коротких текстов, а также методы проектирования программных модулей для мобильных устройств. При рассмотрении возможных решений учитываются как классические словарные и статистические подходы, так и методы машинного обучения, включая трансформерные языковые модели.

Научно-теоретическая и практическая значимость. Научно-теоретическая значимость работы состоит в систематизации подходов к локальному анализу эмоциональной окраски коротких сообщений и в определении факторов, влияющих на применимость таких подходов в условиях мобильного устройства. Практическая значимость заключается в разработке исследовательского прототипа программного модуля, который может быть использован как основа для компонента приватной мобильной клавиатуры и в дальнейшем расширен до инструмента анализа динамики эмоциональной окраски сообщений без передачи исходных сообщений на внешние серверы.

Апробация результатов. Апробация результатов выполнена в форме экспериментальной проверки разработанного прототипа на корпусе коротких русскоязычных сообще-

ний RuGoEmotions, а также демонстрации полного пользовательского сценария на CSV-диалогах. Оценка проводилась по метрикам качества классификации и ресурсным характеристикам моделей.

Глава 1. Анализ предметной области и обзор литературы

1.1. Актуальность и постановка проблемы

Мобильная клавиатура является промежуточным программным компонентом между пользователем и большинством приложений, в которых осуществляется текстовая коммуникация. Поэтому именно клавиатура потенциально получает доступ к наиболее чувствительным фрагментам пользовательского ввода: личным сообщениям, поисковым запросам, фрагментам рабочей переписки, адресам, именам, паролям и иным данным. Одновременно от современной клавиатуры ожидается не только ввод символов, но и интеллектуальная поддержка: исправление опечаток, предсказание следующего слова, персонализация словаря, рекомендации эмодзи и контекстные подсказки. Возникает противоречие между качеством интеллектуальных функций и ограничениями приватности.

Одним из наиболее известных направлений решения этой проблемы является федеративное обучение. В работе Hard и др. [1] рассматривается применение федеративного обучения для предсказания следующего слова в мобильной клавиатуре. Авторы показывают, что обучение рекуррентной языковой модели может быть организовано на пользовательских устройствах без непосредственной выгрузки исходных текстов на сервер. Для темы настоящей работы данная статья важна по двум причинам. Во-первых, она подтверждает практическую значимость сценария мобильной клавиатуры как области применения методов обработки естественного языка. Во-вторых, она демонстрирует, что конкурентоспособность клавиатуры во многом определяется качеством локальной или распределенной адаптации к пользовательскому языку.

Развитие этой идеи представлено в работе Wang и др. [2], посвященной оценке персонализации моделей на устройствах пользователей. Авторы рассматривают не только обучение общей модели, но и вопрос о том, как оценивать пользу персонализации для разных пользователей в условиях, когда данные остаются на устройствах. Для настоящей работы этот подход важен как пример того, что качество мобильной клавиатуры нельзя сводить только к средней точности модели: существенным оказывается индивидуальный эффект для конкретного пользователя, его словаря, стиля общения и типичных сценариев ввода.

Вместе с тем федеративное обучение не является исчерпывающим решением проблемы приватности. В работе Suliman и Leith [3] показано, что при определенных условиях из обновлений федеративной языковой модели, используемой в сценарии предсказания слов для Gboard, могут восстанавливаться фрагменты пользовательского ввода. Данный результат особенно важен для постановки настоящей работы: он показывает, что отсутствие передачи исходного текста на сервер само по себе не гарантирует полной защищенности. Поэтому локальный модуль, выполняющий анализ непосредственно на устройстве и не требующий

отправки ни сообщений, ни обучающих обновлений, представляет самостоятельный интерес.

Таким образом, в литературе уже сформирована проблема приватной интеллектуальной обработки текста в мобильных клавиатурах. Однако основное внимание существующих работ уделяется предсказанию следующего слова и персонализации языковых моделей. Задача локального определения эмоциональной окраски коротких сообщений как вспомогательной функции приватной клавиатуры исследована значительно меньше. Эта ниша и рассматривается в настоящей работе.

1.2. Обзор существующих подходов

Короткие пользовательские сообщения обладают рядом особенностей, которые отличают их от длинных документов и формальных текстов. Они часто состоят из одного или нескольких предложений, содержат опечатки, сокращения, разговорные конструкции, эмодзи, междометия, элементы сленга и неполный контекст. Поэтому методы анализа эмоциональной окраски, хорошо работающие на длинных отзывах или новостных текстах, не всегда напрямую применимы к сообщениям в мессенджерах и мобильной переписке.

Одним из важных ориентиров для исследования коротких эмоционально окрашенных текстов является серия задач SemEval по анализу эмоций в твитах. В работе Baziotis и др. [4] описана система для SemEval-2018 Task 1, в которой использовалась архитектура BiLSTM с механизмом внимания и переносом знаний с близких задач. Сильной стороной такого подхода является способность учитывать порядок слов и выделять значимые фрагменты сообщения. Кроме того, механизм внимания повышает интерпретируемость модели, поскольку позволяет анализировать, какие слова вносят наибольший вклад в предсказание. Ограничение подхода состоит в необходимости обучения нейронной модели и использовании эмбедингов, что усложняет применение на мобильном устройстве по сравнению со словарными или линейными методами.

Другой важный пример обработки коротких сообщений представлен в работе Felbo и др. [5], известной как DeerMoji. Авторы использовали предсказание эмодзи на большом корпусе твитов как форму слабого обучения для получения представлений, полезных при анализе тональности, эмоций и сарказма. Для настоящей работы эта идея особенно близка к пользовательскому сценарию мобильной клавиатуры: эмодзи естественным образом присутствуют в коротких сообщениях и часто выражают эмоциональное состояние автора. В то же время DeerMoji ориентирован на крупномасштабное предварительное обучение и не решает напрямую вопрос компактного локального запуска на смартфоне.

Значимым ресурсом для современных исследований эмоциональной классификации является набор GoEmotions, предложенный Demszky и др. [6]. Он содержит десятки ты-

сяч коротких комментариев, размеченных по 27 категориям эмоций и нейтральному классу. Работа важна тем, что показывает сложность тонкой эмоциональной разметки: даже при использовании BERT-системы среднее значение F1 остается умеренным, что указывает на неоднозначность эмоций в коротких текстах. Для мобильного приложения это означает, что чрезмерно детальная классификация может быть не только вычислительно тяжелой, но и нестабильной с точки зрения качества. Поэтому при проектировании мобильного модуля может быть целесообразно использовать укрупненную схему классов: например, положительная, отрицательная, нейтральная окраска или несколько базовых эмоциональных состояний.

Для русскоязычного исследования важно учитывать не только общие подходы NLP, но и специфику выражения эмоций в русском языке. В лингвистике эмоций Шаховский [23] рассматривает эмотивность как способность языковых единиц выражать эмоциональное состояние и эмоциональное отношение говорящего. Работы Бабенко [24], посвященные русской эмотивной лексике, показывают, что эмоции выражаются не только прямыми названиями чувств, но и широким набором лексических средств. Эти исследования важны для настоящей работы, поскольку обосновывают наличие лингвистической базы у словарных методов, но одновременно указывают на их ограниченность: эмоциональная окраска сообщения может быть связана с контекстом, речевой ситуацией и индивидуальным стилем пользователя.

Современные русскоязычные вычислительные исследования чаще сосредоточены на анализе тональности, оценочных отношений и корпусах, чем на детальной классификации эмоций в личной переписке. В обзоре Kotelnikov [22] показано, что русскоязычные корпуса для sentiment analysis существенно различаются по домену, размеру, схеме разметки и качеству аннотации; следовательно, перенос модели между доменами требует осторожности. Работа Kotelnikova и др. [25] сравнивает словарные методы с RuBERT на русскоязычных корпусах и показывает инженерный компромисс между интерпретируемостью словарей и качеством BERT-подобных моделей. Исследование Razova и др. [26] дополнительно показывает, что RuBERT при решении задач тональности статистически обращает внимание на тональную лексику, что связывает словарные и нейросетевые подходы не как взаимоисключающие, а как частично пересекающиеся.

Отдельное направление русскоязычных работ связано с более сложной структурой оценок. Loukachevitch и Rusnachenko [27] рассматривают sentiment frames для русского языка, где оценка зависит не только от отдельного слова, но и от предиката, участников ситуации и отношений между ними. В задаче RuSentNE-2023 [28] анализируется направленная тональность по отношению к именованным сущностям в русских новостных текстах, а качество оценивается с использованием mazo-усреднения. Для настоящей работы эти исследования важны как подтверждение того, что в русскоязычном NLP существуют домен, схема разметки и баланс классов. При этом они не закрывают задачу локального определения эмоций

в коротких сообщениях мобильной клавиатуры, где текст является приватным, разговорным и привязанным к личной динамике пользователя.

Подходы к эмоциональной классификации можно условно разделить на несколько групп. Первая группа включает словарные методы. Классическим примером является NRC Emotion Lexicon, описанный Mohammad и Turney [8]. В этом подходе словам сопоставляются эмоции и полярность, после чего эмоциональная оценка текста строится на основе присутствующих лексических признаков. Преимущества словарных методов состоят в простоте, интерпретируемости, малом размере и возможности полностью локального запуска. Они хорошо подходят как базовая линия для мобильного модуля. Их слабость заключается в ограниченном учете контекста: словарь плохо обрабатывает отрицания, сарказм, переносный смысл, индивидуальные речевые привычки и новые слова.

Вторая группа включает классические методы машинного обучения и легкие классификаторы. Для представления текста в таких моделях часто используются веса TF-IDF, восходящие к классическим работам по информационному поиску и взвешиванию терминов [9, 10]. Смысл TF-IDF состоит в том, чтобы усиливать признаки, часто встречающиеся в конкретном сообщении, но не являющиеся слишком общими для всего корпуса. Для классификации поверх таких признаков может использоваться логистическая регрессия. Ее статистическая основа восходит к работе Cox [11], а современное изложение многоклассовой логистической регрессии и регуляризации приведено, например, в учебнике Hastie, Tibshirani и Friedman [12]. Для настоящей работы этот класс методов важен тем, что он сочетает воспроизводимость, высокую скорость и достаточно хорошее качество на коротких текстах.

Работа Joulin и др. [13] представляет fastText как эффективный подход к классификации текста, основанный на простых признаках и векторных представлениях. Авторы показывают, что при значительно меньших вычислительных затратах такие модели могут конкурировать с более сложными нейронными архитектурами в ряде задач классификации. Для настоящей работы fastText представляет интерес как практический компромисс между словарным подходом и трансформерной моделью: он способен учитывать статистические закономерности корпуса, но при этом остается достаточно быстрым для локального применения. Дополнительно в работе Joulin и др. [14] рассматривается сжатие моделей классификации, что напрямую связано с ограничениями мобильного запуска.

Третья группа включает рекуррентные нейронные сети и модели с механизмом внимания. Как показано в работе Baziotis и др. [4], BiLSTM с attention может эффективно использовать последовательную структуру короткого текста и выявлять эмоционально значимые слова. Такой подход потенциально лучше словарных методов при обработке контекста, но уступает простым классификаторам по скорости, размеру и сложности реализации. Для мобильной клавиатуры это означает необходимость оценки не только качества классификации, но и задержки ответа, поскольку анализ должен выполняться незаметно для пользователя.

Четвертая группа включает трансформерные модели. Архитектура Transformer была предложена Vaswani и др. [15] и основана на механизме self-attention, позволяющем каждому токеноу учитывать остальные токены последовательности. На этой архитектуре построена модель BERT [16], использующая двунаправленное контекстное представление текста. Для русского языка важны работы по адаптации BERT-подобных моделей к русскоязычным данным, в частности RuBERT [17]. Трансформеры обычно обеспечивают более высокое качество на задачах обработки естественного языка, однако требуют большего объема памяти и вычислений. Для снижения этих требований предложены облегченные варианты. DistilBERT, представленный Sanh и др. [18], использует дистилляцию знаний для уменьшения размера BERT и ускорения вывода при сохранении значительной части качества исходной модели. MobileBERT, предложенный Sun и др. [19], специально ориентирован на устройства с ограниченными ресурсами и демонстрирует возможность применения BERT-подобных архитектур в мобильных сценариях. Эти работы важны для настоящего исследования как примеры того, что трансформерная модель может быть адаптирована к локальному запуску, однако такая адаптация требует отдельной оценки по памяти, времени инференса и энергопотреблению.

Ограничения мобильного запуска подробно рассматриваются в работах, посвященных эффективности трансформеров на устройствах. Paparoulos и др. [20] показывают, что трансформерные архитектуры создают существенную нагрузку на мобильные устройства и требуют программных и аппаратных оптимизаций. Rahman и др. [21] исследуют квантованные реализации трансформерных языковых моделей на edge-устройствах и показывают, что уменьшение разрядности и преобразование модели в формат, пригодный для локального исполнения, позволяют существенно сократить размер модели при умеренной потере качества. Для мобильной клавиатуры эти результаты особенно важны, поскольку модуль анализа эмоций не должен заметно увеличивать расход батареи, занимать значительный объем памяти или создавать задержку при вводе текста.

Таким образом, русскоязычный контекст усиливает необходимость отдельного экспериментального сравнения. Большинство доступных работ и корпусов ориентированы на тональность отзывов, новостей или отношений к сущностям, тогда как сценарий мобильной клавиатуры связан с короткой неформальной коммуникацией, эмодзи, индивидуальным стилем и требованиями приватности. Следовательно, при выборе модели и датасета необходимо учитывать не только общий уровень качества NLP-модели, но и доменную близость к коротким пользовательским сообщениям.

Отдельный вопрос связан не с классификацией отдельного сообщения, а с переходом от последовательности сообщений к оценке эмоциональной окраски текстов за период времени. В психологии близкая постановка рассматривается в рамках ecological momentary assessment. Shiffman, Stone и Hufford [29] описывают подход, при котором состояние челове-

ка оценивается повторно в реальном времени и в естественной среде, что позволяет изучать динамику поведения и переживаний без сильной зависимости от ретроспективной памяти. Для настоящей работы это важно как методологическое обоснование эмоционального календаря: интерес представляет не только отдельное сообщение, но и изменение эмоциональных оценок во времени. При этом в настоящей работе классифицируется не психологическое состояние человека, а эмоциональная окраска отправленного текста.

Близкий пример текстовой агрегации эмоциональных оценок представлен в работе Dodds и др. [30], где по сообщениям Twitter строятся временные ряды коллективного уровня счастья. Масштаб этой работы существенно отличается от персонального календаря мобильной клавиатуры, однако общий принцип совпадает: отдельные текстовые выражения преобразуются в численные эмоциональные оценки, после чего агрегируются по временным интервалам. Это подтверждает применимость временных окон для анализа динамики эмоциональной окраски коротких пользовательских текстов.

Для агрегирования результатов классификации важно учитывать не только итоговую метку, но и распределение оценок по классам. Tumer и Ghosh [31] рассматривают линейные методы комбинирования выходов классификаторов и показывают, что усреднение оценок в пространстве выходов может снижать нестабильность решения. В настоящей работе эта идея используется не для объединения разных классификаторов, а для объединения выходных оценок одной модели по нескольким сообщениям внутри одного временного окна. Такой перенос является инженерным, но содержательно обоснован: среднее распределение сохраняет больше информации, чем голосование по уже выбранным меткам.

При этом выходные оценки модели не следует интерпретировать как калиброванный психологический показатель эмоционального состояния. Guo и др. [32] показывают, что современные нейронные модели могут быть плохо откалиброваны, то есть их уверенность не всегда соответствует реальной вероятности правильного ответа. Поэтому в данной работе confidence используется как относительная оценка уверенности модели и как удобный численный сигнал для агрегации, а не как клиническая или психологическая мера состояния пользователя.

Сравнение рассмотренных работ показывает, что каждая группа методов имеет собственную область применимости. Словарные подходы просты и приватны, но ограничены по качеству. Линейные модели на n-граммных и subword-признаках дают хороший баланс между скоростью и точностью, но зависят от обучающего корпуса. Рекуррентные модели с attention лучше учитывают последовательность слов, однако сложнее для мобильного развертывания. Трансформеры потенциально дают наиболее качественные представления текста, но требуют оптимизации, дистилляции или квантования для локального применения. Для построения эмоционального календаря дополнительно важна возможность агрегировать выходы модели по временным окнам, сохраняя информацию о неопределенности.

1.3. Обоснование выбора моделей для сравнения

Для экспериментальной части работы были выбраны четыре модели, представляющие разные уровни сложности и разные инженерные компромиссы. Такой набор позволяет оценить не только абсолютное качество классификации, но и пригодность модели для локального применения в мобильной клавиатуре. В отличие от серверного сценария, для клавиатуры важны не только точность и полнота, но и задержка ответа, размер артефактов, потребление памяти, простота обновления модели и возможность работы без передачи текста на внешний сервер.

Первым кандидатом является словарный baseline. Он основан на заранее заданных списках эмоционально окрашенных слов и выражений для каждого класса. При обработке сообщения модель подсчитывает совпадения с лексиконом и формирует распределение оценок по классам. Такой подход практически не требует памяти, легко интерпретируется и полностью работает offline. Его включение необходимо как нижняя граница качества: если более сложная модель дает лишь незначительный прирост относительно словаря, ее использование в мобильном модуле может быть неоправданным. Лингвистически такой baseline опирается на представление об эмотивной лексике русского языка [23, 24], а инженерно соответствует группе лексиконных методов, которые остаются важной базовой линией в русскоязычном sentiment analysis [25]. Ограничение словарного подхода состоит в том, что он плохо учитывает контекст, отрицания, переносный смысл, иронию, опечатки и новые разговорные формы.

Вторым кандидатом выбрана модель TF-IDF + Logistic Regression. В ней текст представляется как набор словных и символьных n-грамм, после чего линейный классификатор оценивает принадлежность сообщения к каждому эмоциональному классу. Данный подход является классическим baseline для задач классификации коротких текстов: он быстро обучается, не требует графического ускорителя, имеет небольшой размер и хорошо работает с устойчивыми лексическими маркерами. Использование символьных n-грамм особенно важно для мобильного ввода, поскольку пользовательские сообщения часто содержат опечатки, сокращения, удлинения букв, эмодзи и разговорные формы.

Третьим кандидатом является fastText supervised [13]. В этой модели текст классифицируется на основе представлений слов и их подсловных фрагментов, что позволяет учитывать морфологию, редкие словоформы и часть опечаток. В отличие от TF-IDF + Logistic Regression, fastText не строит разреженное TF-IDF-представление всего словаря, а обучает компактные векторные представления и линейный классификатор поверх усредненного представления сообщения. Поэтому fastText включен как самостоятельный lightweight-кандидат: он существенно проще трансформеров, но методологически отличается от классической логистической регрессии.

Четвертым кандидатом выбран RuBERT-tiny2. Это компактная русскоязычная BERT-подобная модель, предварительно обученная на русскоязычных текстах и затем дообученная на целевой задаче классификации эмоций. Ее преимущество состоит в способности учитывать контекст и порядок слов, а не только наличие отдельных токенов. Русскоязычные исследования показывают, что BERT-подобные модели часто превосходят словарные подходы по качеству, но при этом не полностью отрываются от тональной лексики как источника признаков [25, 26]. Поэтому RuBERT-tiny2 рассматривается как основной нейросетевой кандидат: он должен показывать более высокое качество на неоднозначных сообщениях, где эмоциональная окраска определяется сочетанием слов. При этом даже компактный трансформер имеет заметно больший размер и задержку инференса, поэтому его применимость необходимо проверять экспериментально.

Таким образом, выбранный набор моделей образует шкалу от максимально простого и интерпретируемого решения до компактной трансформерной модели, дообученной на целевой русскоязычной задаче. Это позволяет сделать итоговый выбор не по одному показателю качества, а по балансу между точностью классификации и ресурсной применимостью.

1.4. Выводы по главе

В результате анализа литературы можно сделать вывод, что задача приватной обработки текста в мобильной клавиатуре уже рассматривается в исследованиях, связанных с предсказанием следующего слова, федеративным обучением и персонализацией. Однако даже федеративные подходы не снимают всех рисков приватности, что делает актуальным исследование полностью локальных модулей обработки текста.

Работы по анализу коротких сообщений показывают, что эмоциональная окраска в таких текстах выражается не только лексически, но и через контекст, эмодзи, сокращения и индивидуальный стиль пользователя. Поэтому для решения задачи необходимо сравнивать несколько классов методов: словарные модели, классические классификаторы, рекуррентные нейронные сети и облегченные трансформеры.

Русскоязычные исследования подтверждают важность выбора корпуса, домена и схемы разметки. При этом большая часть работ посвящена тональности, оценочным отношениям или анализу новостных и пользовательских текстов вне сценария мобильной клавиатуры. Настоящая работа занимает промежуточную нишу между исследованиями приватных мобильных клавиатур и исследованиями эмоциональной классификации коротких русскоязычных текстов. Ее основная особенность заключается в ориентации на исследовательский прототип локального lightweight-модуля для будущей мобильной клавиатуры, где необходимо обеспечить баланс между качеством определения эмоциональной окраски, приватностью пользовательских данных и ресурсными ограничениями мобильного устройства.

Глава 2. Постановка задачи и выбор методов решения

2.1. Формальная постановка задачи

В рамках работы решается задача локальной классификации короткого пользовательского сообщения по эмоциональной окраске. На вход модуля поступает текстовое сообщение x , состоящее, как правило, из одного или нескольких коротких предложений. На выходе модуль должен вернуть эмоциональный класс y и набор оценок уверенности по всем рассматриваемым классам. Эти оценки используются для сравнения классов между собой и последующей агрегации по временным окнам, но не интерпретируются как психологическое измерение эмоционального состояния пользователя. В качестве целевой схемы разметки выбраны шесть классов: тревожность, злость, грусть, забота, радость и нейтральное состояние.

Формально модель можно представить как отображение

$$f(x) = \{s_1, s_2, \dots, s_6\}, \quad (2.1)$$

где s_i – выходная оценка уверенности модели для i -го эмоционального класса. Итоговая метка определяется как класс с максимальной оценкой:

$$y = \arg \max_i s_i. \quad (2.2)$$

Для сценария мобильной клавиатуры одной точности классификации недостаточно. Модель должна работать локально, не сохранять исходные сообщения после анализа и иметь малые задержку инференса, размер на диске и потребление оперативной памяти. Поэтому задача выбора модели рассматривается как многокритериальная: необходимо найти компромисс между качеством эмоциональной классификации и пригодностью модели для последующего мобильного применения.

В качестве основной метрики качества используется таско F1, поскольку эмоциональные классы потенциально несбалансированы. Дополнительно оцениваются accuracy, weighted F1, матрица ошибок и средняя уверенность модели на правильных и ошибочных ответах. Для оценки мобильной применимости используются размер модели на диске, медианная задержка инференса, задержка на 95-м процентиле и потребление оперативной памяти.

Выбор модели в работе не сводится к максимизации одной интегральной метрики. Для задачи локального модуля важно одновременно учитывать качество классификации, задержку, размер модели, потребление памяти и сложность внедрения, однако эти критерии имеют разную природу. Их искусственное объединение в одну формулу с произвольными весами может создавать ложное ощущение точности и приводить к методологически спорным выводам. Поэтому в работе используется сравнительный анализ по нескольким критериям без вычисления единого рейтинга.

Процедура выбора устроена следующим образом. Сначала модели сравниваются по качеству классификации на тестовой части RuGoEmotions: основной метрикой является макро F1, дополнительно рассматриваются accuracy, weighted F1, матрица ошибок и качество по отдельным классам. Затем отдельно анализируются ресурсные характеристики: размер модели на диске, задержка инференса на CPU, потребление оперативной памяти и сложность переноса в локальный модуль. После этого выполняется содержательное сравнение: словарная модель рассматривается как нижняя граница и fallback-механизм, RuBERT-tiny2 – как верхняя планка качества, а классические lightweight-модели – как практические кандидаты для локального прототипа.

Такой подход ближе к анализу Парето-компромисса. Модель не считается лучшей только потому, что превосходит остальные по одному показателю. Словарный baseline минимален по размеру и задержке, но заметно проигрывает по качеству, поэтому не подходит как основная модель. RuBERT-tiny2 показывает максимальный макро F1, но требует существенно большего объема хранения и более сложного runtime. fastText оказывается очень быстрым и компактным, однако уступает TF-IDF+LR по качеству. Поэтому для текущего прототипа выбирается TF-IDF+LR: она не доминирует все альтернативы по каждому критерию, но показывает лучший макро F1 среди легких локально запускаемых моделей при приемлемых ресурсных характеристиках. Именно это соотношение, а не значение заранее заданной весовой функции, используется как основание для выбора модели прототипа.

Для компактного представления результатов в таблицах далее используются сокращенные названия моделей, расшифрованные в таблице 2.1.

Таблица 2.1 – Сокращенные названия моделей

Название	Расшифровка
Dict	словарный baseline
TF-IDF+LR	TF-IDF + Logistic Regression
fastText	нативная fastText supervised модель с подсловными признаками
RuBERT-tiny2	дообученная русскоязычная BERT-подобная модель

Таблица 2.2 – Ресурсные характеристики сравниваемых моделей

Модель	P50, мс	P95, мс	Размер, МБ	RAM, МБ	Простота внедрения
Dict	0,065	0,067	0,002	133,098	высокая
TF-IDF+LR	0,305	0,325	3,359	151,746	высокая
fastText	0,261	0,264	6,491	143,785	средняя
RuBERT-tiny2	5,592	7,558	225,175	1076,141	средняя

2.2. Математический аппарат и обоснование выбранных методов

Для сравнения выбраны четыре группы моделей, отражающих разные уровни сложности и разные компромиссы между качеством и ресурсоемкостью.

Все рассматриваемые методы решают одну и ту же задачу многоклассовой классификации. Пусть x – короткое текстовое сообщение, а

$$E = \{e_1, e_2, \dots, e_K\}$$

– множество эмоциональных классов, где в настоящей работе $K = 6$. Требуется построить функцию

$$f : X \rightarrow E,$$

которая сопоставляет сообщению одну из эмоций. Более информативная постановка состоит не только в выборе метки, но и в оценке распределения

$$p(e \mid x), \quad e \in E,$$

то есть модельной оценки уверенности для каждого эмоционального класса. Итоговая метка выбирается по максимуму этой оценки:

$$\hat{e} = \arg \max_{e \in E} p(e \mid x).$$

Именно это правило используется во всех моделях проекта: различается только способ получения оценок $p(e \mid x)$.

Для обучаемых моделей используется принцип минимизации эмпирического риска. Если обучающая выборка имеет вид

$$D = \{(x_i, y_i)\}_{i=1}^n,$$

где y_i – истинная эмоциональная метка сообщения x_i , то параметры модели θ подбираются из условия

$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n \ell(y_i, p_{\theta}(\cdot \mid x_i)) + \lambda \Omega(\theta).$$

Здесь ℓ – функция потерь, $\Omega(\theta)$ – регуляризатор, а λ задает силу регуляризации. Для вероятностной многоклассовой классификации стандартной функцией потерь является кросс-энтропия:

$$\ell(y_i, p_{\theta}(\cdot \mid x_i)) = -\log p_{\theta}(y_i \mid x_i).$$

Минимизация этой функции эквивалентна максимизации правдоподобия правильных классов на обучающей выборке. Поэтому логистическая регрессия, fastText и дообучаемый RuBERT-tiny2 различаются прежде всего способом построения признакового представления сообщения, но обучаются в рамках близкой вероятностной схемы.

Для оценки качества используется F1-мера, основанная на precision и recall. Для класса e

$$\text{Precision}_e = \frac{TP_e}{TP_e + FP_e}, \quad \text{Recall}_e = \frac{TP_e}{TP_e + FN_e},$$

где TP_e, FP_e, FN_e – число истинно положительных, ложноположительных и ложноотрицательных предсказаний для класса e . Тогда

$$F1_e = \frac{2 \cdot \text{Precision}_e \cdot \text{Recall}_e}{\text{Precision}_e + \text{Recall}_e}.$$

Основной метрикой выбрана macro F1:

$$\text{MacroF1} = \frac{1}{K} \sum_{e \in E} F1_e.$$

Она усредняет качество по классам без учета их частот и поэтому лучше подходит для эмоциональной классификации, где важно не потерять менее очевидные или хуже распознаваемые эмоции.

Первый кандидат – словарный baseline. Он основан на наборе ключевых слов и морфем, связанных с каждым эмоциональным классом. Пусть E – множество эмоциональных классов, L_e – словарь маркеров для эмоции $e \in E$, а $T(x)$ – множество токенов и нормализованных фрагментов сообщения x . Тогда простейшая оценка класса может быть задана как сумма совпадений:

$$c_e(x) = \sum_{t \in T(x)} w_t \cdot I(t \in L_e),$$

где $I(\cdot)$ – индикатор совпадения, а w_t – вес маркера. В простом варианте все веса равны единице, но для эмоционально более сильных слов можно задавать больший вес. Чтобы получить распределение оценок по классам и избежать нулей, используется сглаживание:

$$p(e | x) = \frac{c_e(x) + \alpha}{\sum_{e' \in E} (c_{e'}(x) + \alpha)}.$$

Словарный метод почти не требует памяти, легко интерпретируется и полностью работает offline. Он включен как нижняя граница качества и как пример максимально легкого решения. Его ограничение состоит в том, что совпадение отдельных слов плохо учитывает порядок слов, отрицания, сарказм, переносный смысл и индивидуальные речевые привычки. Например, фраза «не злюсь» может содержать маркер злости, но по смыслу выражать отрицание этой эмоции.

Второй кандидат – модель TF-IDF на словных и символьных n-граммах с логистической регрессией. На первом этапе сообщение преобразуется в разреженный вектор признаков. Если $tf(t, d)$ – частота термина t в документе d , $df(t)$ – число документов корпуса, содержащих t , а N – общее число документов, то сглаженный TF-IDF-вес можно записать как

$$\text{tfidf}(t, d) = tf(t, d) \cdot \left(\log \frac{N + 1}{df(t) + 1} + 1 \right).$$

В настоящей работе в качестве термов используются словные n-граммы и символьные n-граммы. Словные n-граммы отражают устойчивые выражения, например «очень рада» или «мне страшно», а символьные n-граммы частично компенсируют опечатки, разговорные формы, неполные слова и морфологические варианты. После векторизации сообщение представляется как вектор $x \in \mathbb{R}^m$, где m – размерность пространства признаков.

Для классификации используется многоклассовая логистическая регрессия. Для каждого класса e обучаются веса w_e и свободный член b_e , после чего вероятность класса оценивается через softmax:

$$p(e | x) = \frac{\exp(w_e^\top x + b_e)}{\sum_{e' \in E} \exp(w_{e'}^\top x + b_{e'})}.$$

Параметры модели подбираются путем минимизации регуляризованной кросс-энтропии:

$$\mathcal{L}(W, b) = - \sum_{i=1}^n \log p(y_i | x_i) + \lambda \|W\|_2^2.$$

Регуляризация ограничивает величину весов и снижает риск переобучения на отдельных редких n-граммах. Такой подход является сильным классическим baseline: он быстро обучается, хорошо работает на коротких текстах с явными лексическими маркерами и сохраняется в компактном формате `joblib`. При этом модель остается линейной: она не строит глубокое контекстное представление сообщения и хуже обрабатывает случаи, где эмоция определяется сложным сочетанием слов.

Третий кандидат – нативная модель `fastText supervised` [13]. В реализации обучающий набор преобразуется в формат `__label__<класс> <текст>`, после чего `fastText` обучает компактные представления слов и подсловных фрагментов. Пусть $G(x)$ – множество словных и подсловных признаков сообщения x , а v_g – обучаемый вектор признака g . Тогда представление сообщения можно описать как усреднение его признаков:

$$z(x) = \frac{1}{|G(x)|} \sum_{g \in G(x)} v_g.$$

Классификация выполняется линейным слоем и softmax-функцией:

$$p(e | x) = \frac{\exp(u_e^\top z(x) + a_e)}{\sum_{e' \in E} \exp(u_{e'}^\top z(x) + a_{e'})}.$$

Обучение, как и в логистической регрессии, сводится к минимизации отрицательного логарифма вероятности правильного класса. Главное отличие `fastText` от TF-IDF+LR состоит в представлении текста: вместо фиксированного разреженного вектора TF-IDF модель обучает плотные векторные представления признаков. Использование подсловных n-грамм позволяет разделять информацию между родственными словоформами и частично справляться с опечатками. В эксперименте использовались словные биграммы, размерность векторов 100, подсловные n-граммы длиной от 3 до 6 символов и softmax-функция потерь. После обучения модель сохраняется в формате `.ftz` либо `.bin`. Такой вариант выбран как отдельный

lightweight-кандидат: он сохраняет идею устойчивости к словоформам и опечаткам, но не является второй версией TF-IDF + Logistic Regression.

Четвертый кандидат – RuBERT-tiny2, дообученный на выбранной схеме из шести эмоций. Эта модель относится к BERT-подобным трансформерным энкодерам [16, 17]. В отличие от словарных и линейных моделей, трансформер строит контекстные представления токенов: один и тот же токен может получать разные векторы в зависимости от соседних слов. Основной операцией является self-attention [15]. Для матриц запросов Q , ключей K и значений V внимание вычисляется как

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_k}} \right) V,$$

где d_k – размерность ключей. Благодаря этому каждый токен сообщения может учитывать другие токены, что особенно важно для отрицаний, уточнений и эмоционально неоднозначных фраз.

При дообучении на задачу классификации сообщение токенизируется, пропускается через энкодер, после чего представление специального классификационного токена используется как вектор всего сообщения h_{cls} . Вероятности классов вычисляются линейным классификатором:

$$p(e | x) = \frac{\exp(q_e^\top h_{\text{cls}} + r_e)}{\sum_{e' \in E} \exp(q_{e'}^\top h_{\text{cls}} + r_{e'})}.$$

Дообучение выполняется минимизацией кросс-энтропии на размеченных сообщениях. RuBERT-tiny2 рассматривается как основной нейросетевой кандидат, поскольку она ориентирована на русский язык и существенно компактнее полноразмерных BERT-моделей. Для мобильного применения предполагается последующий экспорт в ONNX и квантование до INT8, что соответствует подходам дистилляции и сжатия, рассмотренным в работах по DistilBERT и MobileBERT [18, 19]. Ограничением модели являются больший размер, более сложная инфраструктура запуска и более высокая задержка по сравнению с TF-IDF+LR и fastText.

Таким образом, выбранный набор моделей позволяет сравнить четыре уровня сложности: словарь, классическое машинное обучение, subword-классификатор и трансформерную модель. Это делает итоговый выбор не формальным сравнением точности, а инженерной оценкой применимости модели в ограничениях локального пользовательского сценария.

2.3. Выводы по главе

Во второй главе была сформулирована задача локальной классификации коротких сообщений по шести эмоциональным классам и определены требования к модели для мобильной клавиатуры. Показано, что итоговый выбор не должен сводиться к одной искусственной

интегральной формуле: качество классификации, задержка, размер и память рассматриваются как отдельные критерии. Для дальнейшей экспериментальной проверки выбраны словарный baseline, TF-IDF с логистической регрессией, fastText supervised и RuBERT-tiny2.

Глава 3. Реализация и экспериментальная проверка

3.1. Описание реализации

Для проверки предложенного подхода был разработан исследовательский Python-прототип `emotion_keyboard`. Он имитирует ключевой сценарий будущего локального компонента мобильной клавиатуры: принимает текст сообщения, выполняет нормализацию, получает оценки уверенности по эмоциональным классам и затем агрегирует результаты по временным окнам. Исходные сообщения после анализа не требуются для построения календаря; для итогового представления достаточно сохранить дату, период дня, отправителя, распределение оценок по классам и количество сообщений.

Реализация разделена на несколько файлов, каждый из которых отвечает за отдельный этап эксперимента или демонстрационного сценария:

- `labels.py` задает фиксированный список шести целевых классов: тревожность, злость, грусть, забота, радость и нейтральное состояние;
- `preprocess.py` выполняет нормализацию текста: приведение к нижнему регистру, замену буквы «ё», нормализацию пробелов, ссылок, упоминаний и части эмодзи;
- `prepare_rugoemotions.py` преобразует RuGoEmotions к выбранной шестиклассовой схеме, формирует одноклассовую разметку, балансирует данные и сохраняет `train.csv`, `val.csv`, `test.csv`;
- `models.py` содержит единый интерфейс моделей: словарную модель, sklearn-модель, fastText-модель и transformer-модель, у которых есть общий метод `predict_proba`;
- `train_baselines.py` обучает словарный baseline, TF-IDF + Logistic Regression и fastText supervised;
- `train_transformer.py` выполняет дообучение RuBERT-tiny2 и содержит процедуру экспорта модели в ONNX;
- `benchmark.py` применяет все сравниваемые модели к тестовой выборке, считает accuracy, macro F1, weighted F1, задержку, размер модели, RSS-память и сохраняет матрицу ошибок выбранной модели;
- `calendar.py` реализует усреднение выходных оценок модели для набора сообщений;
- `analyzer.py` предоставляет класс `EmotionAnalyzer`, то есть публичный интерфейс для загрузки выбранной TF-IDF+LR-модели из `models/tfidf_lr` и применения ее к новым сообщениям;

- `dialog_calendar.py` реализует демонстрационный сценарий: читает CSV-диалоги, добавляет предсказания и оценки по классам, делит сообщения по отправителям, датам и периодам дня, после чего сохраняет широкие календарные таблицы и сводный файл `calendar_summary.csv`.

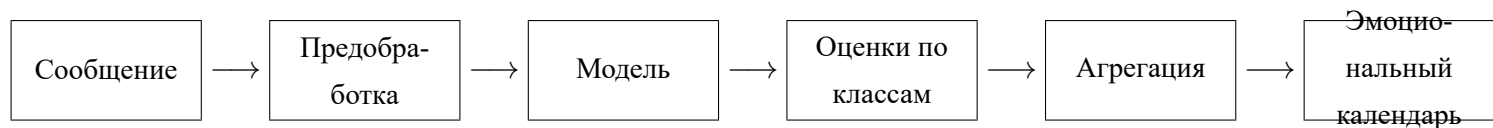


Рис. 3.1 – Общая схема работы исследовательского прототипа локального анализа эмоциональной окраски сообщений

Публичный интерфейс модуля имеет следующий вид:

```

analyzer = EmotionAnalyzer.load("models/tfidf_lr")
result = analyzer.predict("я сегодня очень устала, но рада")
day = analyzer.aggregate_day(messages)
  
```

В результате вызова `predict` возвращаются метка класса, уверенность модели и оценки по всем шести классам. Вызов `aggregate_day` возвращает доминирующую эмоцию дня, число обработанных сообщений и распределение оценок по эмоциональным классам. Для демонстрации календаря используется более полный сценарий из `dialog_calendar.py`: входной CSV-диалог преобразуется в таблицу с предсказаниями, затем в сводную таблицу по отправителям, датам и периодам дня. Такая схема соответствует требованию приватности: прототип может служить основой для локального компонента клавиатуры и не требует передачи исходных сообщений на сервер.

3.2. Эксперименты и результаты

В рамках основного эксперимента был использован корпус RuGoEmotions [7]. Выбор данного набора данных обусловлен несколькими причинами. Во-первых, он является русскоязычной адаптацией задачи *fine-grained emotion classification* и поэтому ближе к целевому сценарию работы, чем англоязычные наборы без перевода. Во-вторых, сообщения в корпусе имеют пользовательский характер и по стилю ближе к коротким репликам, комментариям и фрагментам переписки, чем к длинным литературным или новостным текстам. В-третьих, корпус подходит для шестиклассовой схемы эмоциональной разметки, выбранной в данной работе, и позволяет провести воспроизводимое сравнение моделей на русскоязычных коротких сообщениях. RuGoEmotions выбран не как универсальный источник для всех видов русскоязычной коммуникации, а как наиболее близкий из доступных ресурсов к задаче многоклассовой классификации эмоций; многие русскоязычные корпуса, рассмотренные

в литературе, ориентированы на тональность отзывов, новостей или оценку отношения к сущностям, что хуже соответствует личной переписке.

Исходная разметка RuGoEmotions содержит более детальные эмоциональные категории, чем требуется в модуле мобильной клавиатуры. Для пользовательского интерфейса эмоционального календаря слишком большое число классов может быть неудобным: отдельные близкие эмоции сложнее интерпретировать, а качество классификации на коротких сообщениях становится менее устойчивым. Поэтому исходные эмоции были агрегированы в шесть укрупненных классов: тревожность, злость, грусть, забота, радость и нейтральное состояние. Точное соответствие исходных меток целевым классам приведено в таблице 3.1.

Таблица 3.1 – Соответствие исходных меток RuGoEmotions целевым классам

Целевой класс	Исходные метки RuGoEmotions
тревожность	fear, nervousness, embarrassment
злость	anger, annoyance, disapproval, disgust
грусть	disappointment, grief, remorse, sadness
забота	caring, gratitude, love
радость	admiration, amusement, approval, excitement, joy, optimism, pride
нейтрально	neutral

Поскольку часть записей в исходном корпусе может иметь несколько эмоциональных меток, для обучения модуля была сформирована одноклассовая постановка. Если у сообщения присутствовало несколько меток, сначала они переводились в укрупненные классы. Затем выбирался первый активный класс в фиксированном порядке целевой схемы: тревожность, злость, грусть, забота, радость, нейтрально. При этом нейтральная метка при наличии эмоциональной метки исключалась из списка кандидатов, так как в задаче эмоционального календаря важнее сохранить выраженную эмоциональную окраску сообщения. Такой подход упрощает постановку задачи до многоклассовой классификации и делает результаты разных моделей сопоставимыми. Данное правило является эвристическим и может влиять на итоговое распределение классов; более строгим вариантом дальнейшего развития является сохранение multi-label постановки или экспертная настройка приоритетов между эмоциональными категориями.

После преобразования разметки корпус был сбалансирован по классам. Балансировка необходима, поскольку для эмоциональных данных характерна неравномерность: нейтральные и часто встречающиеся положительные реакции могут существенно превосходить редкие классы. Без балансировки модель могла бы получать высокую ассигуру за счет предпочтения частых классов, но при этом плохо распознавать тревожность, грусть или злость. В данной работе приоритетной метрикой является macro F1, поскольку она одинаково учитывает качество по каждому классу и снижает влияние дисбаланса. Такой выбор согласует-

ся с практикой русскоязычных benchmark-задач, например RuSentNE-2023 [28], где макро-усреднение используется для оценки качества в условиях нескольких классов.

Итоговый набор содержит 7200 сообщений, по 1200 сообщений на каждый из шести классов. Данные были разделены на три непересекающиеся части: 4608 сообщений в обучающей выборке, 1152 сообщения в валидационной выборке и 1440 сообщений в тестовой выборке. Такое разделение соответствует соотношению 64/16/20 и сохраняет баланс классов внутри каждого split: в обучающей части используется по 768 сообщений каждого класса, в валидационной – по 192 сообщения, в тестовой – по 240 сообщений. Обучающая выборка применяется для настройки параметров моделей, валидационная – для промежуточного контроля качества и выбора конфигурации, а тестовая используется только для итоговой оценки. Это разделение снижает риск переоценки качества модели и делает эксперимент воспроизводимым.

Бенчмарк качества и ресурсных характеристик выполнялся в одном Python-окружении. Для оценки качества каждая модель применялась к полной тестовой выборке, после чего вычислялись accuracy, macro F1 и weighted F1. Для оценки задержки инференса применялся вызов `predict_proba`: для классических моделей использовались 240 сообщений и 7 повторов, для трансформерных моделей – 48 сообщений и 3 повтора; итоговое время делилось на число обработанных сообщений. Потребление памяти фиксировалось как RSS всего Python-процесса во время бенчмарка, поэтому данный показатель отражает не только размер весов модели, но и накладные расходы используемых библиотек.

Таблица 3.2 – Состав экспериментальных данных после балансировки

Часть данных	Доля	Всего сообщений	Сообщений на класс
Train	64%	4608	768
Validation	16%	1152	192
Test	20%	1440	240
Итого	100%	7200	1200

В таблице 3.3 приведены результаты качества для четырех кандидатов из протокола отбора. Для lightweight-моделей указаны результаты локального обучения на RuGoEmotions. RuBERT-tiny2 был дообучен на сформированной обучающей выборке в течение 3 эпох с максимальной длиной последовательности 96 токенов, $\text{batch size} = 8$ и $\text{learning rate} = 2 \cdot 10^{-5}$. Обучение выполнялось при фиксированном $\text{random seed} = 42$; на каждой эпохе качество контролировалось на валидационной выборке, а итоговая оценка проводилась только на тестовой части. Значения для RuBERT-tiny2 соответствуют отдельному ранее выполненному transformer-запуску, тогда как lightweight-модели были пересчитаны в текущем воспроизводимом запуске. Ресурсные характеристики этих же моделей приведены отдельно в таблице 2.2, поскольку качество и вычислительная применимость не объединяются в одну искус-

ственную метрику.

Таблица 3.3 – Результаты экспериментального сравнения моделей на тестовой части

RuGoEmotions			
Модель	Accuracy	Macro F1	Weighted F1
Dict	0,292	0,259	0,259
TF-IDF+LR	0,422	0,423	0,423
fastText	0,398	0,393	0,393
RuBERT-tiny2	0,478	0,473	0,473

По результатам эксперимента наибольшее качество классификации показал RuBERT-tiny2: его macro F1 равен 0,473. Это ожидаемый результат, поскольку трансформерная модель использует контекстные представления и лучше учитывает сочетания слов, порядок токенов и неоднозначность коротких сообщений. Поэтому RuBERT-tiny2 в данной работе рассматривается как наиболее сильная модель по качеству и как ориентир для дальнейшего развития системы.

Словарный baseline имеет минимальный размер и задержку, но его macro F1 равен только 0,259. Поэтому словарь не рассматривается как основная модель, несмотря на привлекательные ресурсные характеристики. Его роль в эксперименте – задать нижнюю границу качества и показать, что простого набора эмоциональных слов недостаточно для анализа пользовательских сообщений: такой подход плохо учитывает контекст, отрицания, переносный смысл и индивидуальные речевые формулировки.

TF-IDF + Logistic Regression показала лучший результат среди легких локально запускаемых моделей: macro F1 = 0,423. Это объясняется тем, что сочетание словных и символьных n-грамм хорошо подходит для коротких сообщений: модель улавливает устойчивые лексические маркеры эмоций, разговорные формы и часть опечаток. При этом она сохраняет приемлемую задержку инференса и небольшой размер на диске, поэтому является сильным практическим кандидатом для прототипа.

fastText supervised получил macro F1 = 0,393. По качеству он уступил TF-IDF + Logistic Regression, однако оказался быстрее: P95 задержки составил 0,264 мс, а размер модели – 6,491 МБ. Этот результат показывает, что fastText является не заменой логистической регрессии, а отдельным вариантом компромисса: он особенно интересен в случае более жестких ограничений по скорости, но в текущем эксперименте проигрывает по основной метрике качества.

Таким образом, максимальное качество и минимальные ресурсные требования достигаются разными моделями. Словарь слишком слаб по качеству, RuBERT-tiny2 является лучшим по macro F1, но требует более тяжелой инфраструктуры, fastText выигрывает у TF-IDF+LR по скорости, но уступает по качеству, а TF-IDF+LR после ограничения редких n-

грамм остается компактной. Поэтому для реализации текущего прототипа эмоционального календаря выбрана TF-IDF + Logistic Regression: она не является абсолютным лидером по качеству, но среди легких моделей дает наилучший макро F1 при приемлемых ресурсных характеристиках.

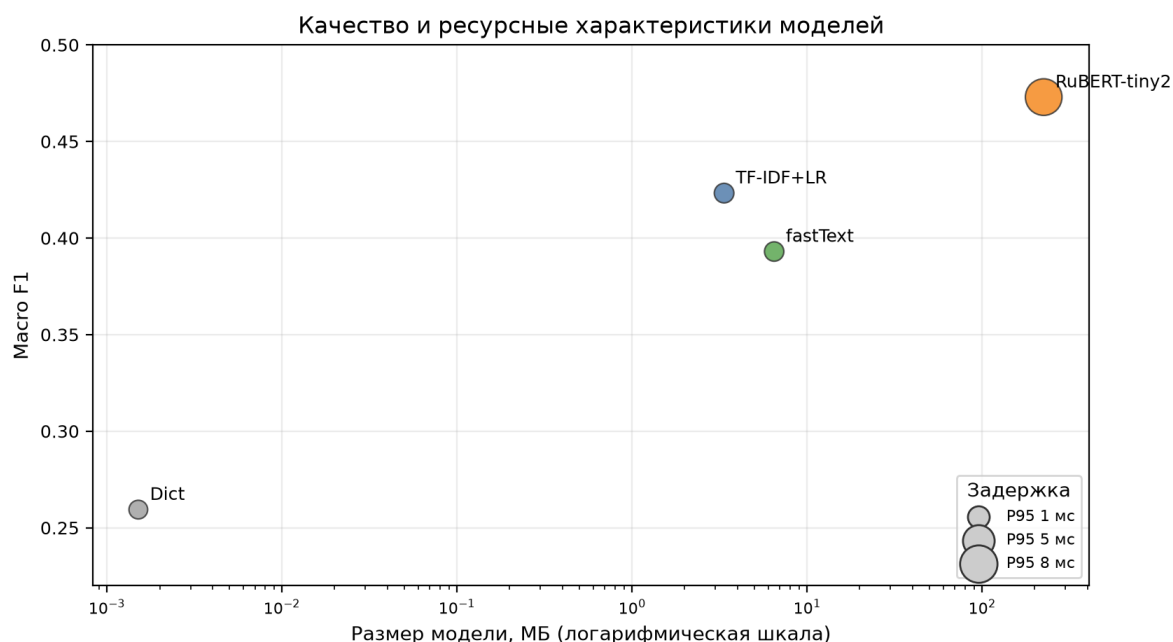


Рис. 3.2 – Сравнение качества и ресурсных характеристик моделей

На рисунке 3.2 визуально показано соотношение качества и ресурсных характеристик. Словарный baseline расположен в области минимального размера и задержки, но заметно уступает по макро F1. RuBERT-tiny2 расположен в области максимального качества, но имеет наибольший размер. fastText занимает область быстрых моделей, однако по качеству уступает TF-IDF+LR. TF-IDF+LR имеет большую задержку, чем fastText, но меньший размер и лучший результат среди lightweight-кандидатов по макро F1. Такой вид графика подчеркивает, что выбор модели является не результатом подгонки весов, а анализом видимого компромисса между качеством и ресурсами.

Помимо агрегированного значения макро F1, для выбранной модели были рассчитаны precision, recall и F1-score по каждому классу. Это позволяет определить, какие эмоциональные категории распознаются устойчиво, а какие чаще смешиваются с другими классами. Результаты приведены в таблице 3.4.

Матрица ошибок выбранной модели приведена в таблице 3.5. Она показывает, что наиболее устойчиво распознаются классы «забота» (145 правильных предсказаний из 240), «тревожность» (111 из 240) и «радость» (103 из 240). Наиболее сложными оказались классы «нейтрально», «злость» и «грусть», между которыми часто возникают взаимные ошибки. Например, нейтральные сообщения нередко классифицируются как злость или радость из-

Таблица 3.4 – Качество выбранной модели по классам

Класс	Precision	Recall	F1-score
тревожность	0,500	0,463	0,481
злость	0,327	0,346	0,336
грусть	0,400	0,392	0,396
забота	0,647	0,604	0,625
радость	0,375	0,429	0,400
нейтрально	0,309	0,296	0,302

за отдельных эмоционально окрашенных слов, а злость может смешиваться с тревожностью и нейтральным классом. Это ожидаемо для коротких пользовательских сообщений: одна реплика может одновременно содержать оценочность, бытовой контекст и эмоционально нейтральную информацию. Таким образом, результат не является идеально делимым и не подходит для точной эмоциональной диагностики, однако достаточен для демонстрации исследовательского прототипа и сравнения lightweight-подходов в одинаковых условиях.

Таблица 3.5 – Матрица ошибок выбранной модели

Истинный / предсказанный класс	трев.	зл.	грусть	забота	рад.	нейтр.
тревожность	111	28	39	11	25	26
злость	31	83	31	16	38	41
грусть	37	33	94	15	30	31
забота	10	16	19	145	33	17
радость	9	37	19	28	103	44
нейтрально	24	57	33	9	46	71

Отдельно был проверен пример работы публичного интерфейса. Для сообщения «я сегодня очень устала, но рада, что все получилось» модуль возвращает эмоциональную метку, уверенность и распределение оценок по всем классам. При агрегации нескольких сообщений за день модуль возвращает число сообщений, доминирующую эмоцию и распределение оценок по классам. Это демонстрирует возможность построения эмоционального календаря без хранения исходных текстов.

3.3. Построение эмоционального календаря по диалогам

После выбора оптимальной модели был реализован модуль построения эмоционального календаря по CSV-диалогу. В качестве входных данных используются демонстрационные реалистичные примеры переписки за период с 01.05.26 по 05.05.26: диалог Александра с Екатериной и диалог Анастасии с мамой. Каждая запись входной таблицы содержит дату, время отправки, отправителя и текст сообщения. Такой формат близок к тому, как мобильная

клавиатура могла бы обрабатывать поток отправляемых сообщений на устройстве пользователя. Сама идея календаря опирается на подход повторных измерений состояния во времени, близкий к ecological momentary assessment [29], и на исследования временной динамики эмоциональных оценок по текстовым данным [30].

На первом этапе модуль загружает выбранную TF-IDF+LR-модель из каталога `models/tfidf_lr` и применяет ее к каждому сообщению. Для каждой строки исходного диалога добавляются предсказанная эмоция, уверенность модели и шесть колонок с оценками по классам: тревожность, злость, грусть, забота, радость и нейтральное состояние. Таким образом, исходный текст преобразуется в набор числовых признаков, которые затем могут быть агрегированы без необходимости хранить сами сообщения.

На втором этапе сообщения разделяются по отправителю. В рассматриваемых демонстрационных данных формируются четыре календаря: для Александра, Екатерины, Анастасии и мамы. Далее сообщения каждого отправителя делятся по датам и трем периодам дня: утро, день и вечер. Утром считается интервал с 06:00 до 11:59, днем – с 12:00 до 17:59, вечером – с 18:00 до 23:59. Если сообщение попадает в интервал с 00:00 до 05:59, оно также относится к вечернему периоду, чтобы не вводить отдельную ночную категорию в демонстрационном календаре.

Для определения настроения периода используется не голосование по итоговым меткам сообщений, а усреднение оценок модели по классам. Такой выбор связан с тем, что усреднение выходных оценок сохраняет информацию о близких по величине классах и соответствует общему принципу линейного комбинирования выходов классификаторов [31]. Пусть в некотором периоде находится n сообщений, а модель для сообщения i возвращает оценку $s_i(e)$ для эмоции e . Тогда средняя оценка эмоции за период вычисляется по формуле

$$\bar{s}(e) = \frac{1}{n} \sum_{i=1}^n s_i(e).$$

Итоговой эмоцией периода выбирается класс

$$e^* = \arg \max_e \bar{s}(e).$$

Уверенность периода равна значению $\bar{s}(e^*)$. Такой способ агрегации выбран потому, что он сохраняет степень неопределенности модели. Простое голосование учитывало бы только итоговую метку каждого сообщения и теряло бы информацию о близких по величине оценках. Для коротких сообщений это особенно важно: одна реплика может содержать смешанную эмоциональную окраску, а разница между двумя наиболее высокими оценками может быть небольшой. При интерпретации confidence учитывается, что выходные оценки классификаторов могут быть неидеально откалиброваны [32]; поэтому в модуле они используются как относительные оценки уверенности для сравнения классов и периодов, а не как точные психологические измерения.

Результатом работы модуля являются две таблицы с предсказаниями для исходных диалогов и четыре таблицы эмоционального календаря. В календаре каждая дата делится на три периода: утро, день и вечер. Для каждого периода сохраняются итоговая эмоция, уверенность и количество сообщений. Если в некотором периоде нет сообщений, ячейка заполняется значением «нет данных». В демонстрационном запуске для Александра, Екатерины, Анастасии и мамы были построены отдельные календарные таблицы за пять дней; дополнительные агрегированные данные сохранены в сводной таблице `calendar_summary.csv`.

Для иллюстрации пользовательского сценария в таблице 3.6 приведен календарь для одного отправителя. Таблица демонстрирует не качество психологической диагностики, а формат агрегированного представления эмоциональной окраски сообщений по периодам дня. В каждой ячейке указаны итоговая эмоция, уверенность и число сообщений.

Таблица 3.6 – Пример эмоционального календаря для Александра

Дата	Утро	День	Вечер
01.05.26	злость / 0,203 / 17 сообщ.	злость / 0,217 / 19 сообщ.	радость / 0,198 / 21 сообщ.
02.05.26	нейтрально / 0,220 / 17 сообщ.	грусть / 0,196 / 24 сообщ.	забота / 0,258 / 18 сообщ.
03.05.26	тревожность / 0,198 / 16 сообщ.	радость / 0,183 / 25 сообщ.	забота / 0,290 / 17 сообщ.
04.05.26	забота / 0,197 / 17 сообщ.	радость / 0,187 / 24 сообщ.	радость / 0,203 / 18 сообщ.
05.05.26	нейтрально / 0,209 / 17 сообщ.	злость / 0,230 / 25 сообщ.	нейтрально / 0,193 / 17 сообщ.

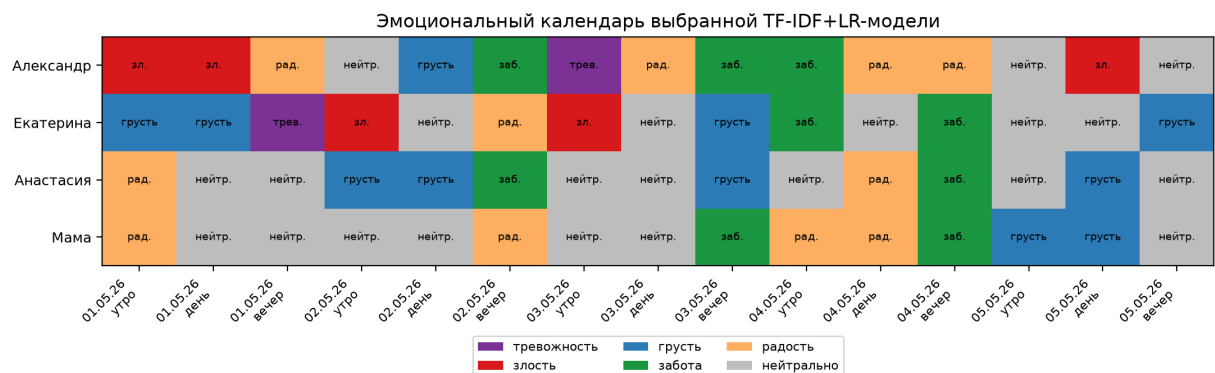


Рис. 3.3 – Визуализация эмоционального календаря для выбранной TF-IDF+LR-модели

На рисунках 3.3 и 3.4 тот же результат представлен в более наглядном виде: первая тепловая карта показывает доминирующую эмоцию в каждой ячейке календаря, вторая – значение `confidence` для этой ячейки. Низкие значения `confidence` дополнительно подчеркивают, что результат следует интерпретировать как агрегированную оценку эмоциональной окраски сообщений, а не как точное психологическое измерение.

3.4. Анализ результатов

Полученный результат показывает, что даже простые методы могут быть полезны как первая версия локального модуля, если эмоциональные классы выражены явно. Словарный

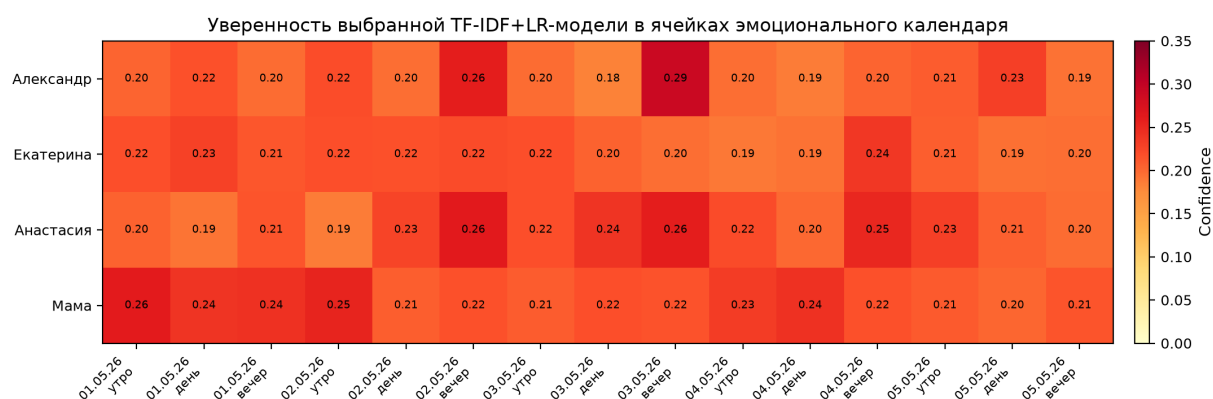


Рис. 3.4 – Уверенность выбранной TF-IDF+LR-модели в ячейках эмоционального календаря

baseline выигрывает по размеру, скорости и интерпретируемости, что делает его удобной нижней границей и возможным fallback-механизмом для мобильной клавиатуры.

Дополнительно к формальному тестированию на RuGoEmotions в работе используются два реалистичных диалога: диалог Анастасии с мамой и диалог Александра с Екатериной. Эти данные не являются независимым размеченным тестовым набором в строгом смысле, однако они полезны для продуктовой проверки: на них можно анализировать динамику эмоциональной окраски сообщений во времени, устойчивость модели к длинным репликам и пригодность результатов для эмоционального календаря. В отличие от тестовой выборки RuGoEmotions, такие диалоги демонстрируют не только качество отдельной классификации, но и работу всего пользовательского сценария: сообщение – оценки эмоций – агрегация по времени – календарная таблица. При этом результаты на RuGoEmotions нельзя напрямую считать гарантией качества на любых русскоязычных мессенджерных данных: стиль общения, длина реплик, частота эмодзи и распределение эмоций могут отличаться у разных пользователей.

Для визуальной проверки пользовательского сценария были построены графики на основе агрегированных выходов моделей. Рисунок 3.5 не заменяет формальную оценку качества на RuGoEmotions, однако показывает, как разные модели интерпретируют одни и те же диалоги. Например, словарный baseline чаще относит сообщения к нейтральному классу, тогда как обученные модели дают более распределенную картину эмоциональной окраски. Это подтверждает практическую пользу визуализации: она позволяет увидеть не только итоговую метку, но и характер поведения модели на демонстрационной последовательности сообщений.

С точки зрения будущего мобильного внедрения разработанный прототип удовлетворяет ключевому требованию приватности: анализ выполняется локально, а исходные сообщения не сохраняются после классификации. Для эмоционального календаря достаточно хранить агрегированные значения по дням: количество сообщений и распределение оценок

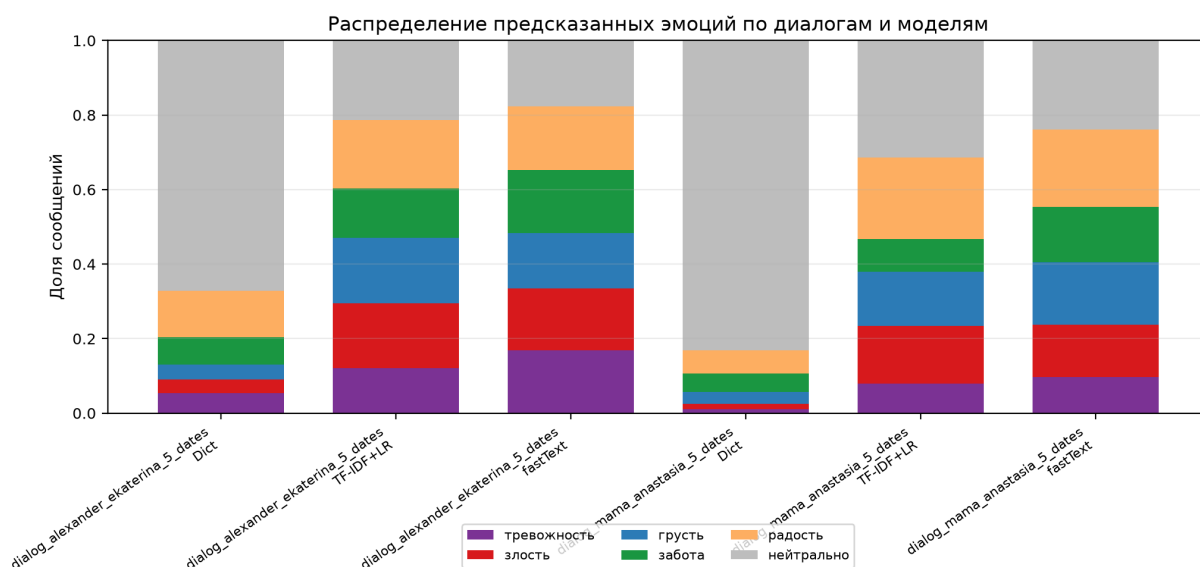


Рис. 3.5 – Распределение предсказанных эмоций в демонстрационных диалогах для разных моделей

по эмоциональным классам.

Полученные результаты показывают, что выбранная модель пригодна для демонстрации локального сценария анализа эмоциональной окраски сообщений, однако ее качество остается ограниченным. Значение $\text{macro F1} = 0,423$ не позволяет использовать модель как надежный инструмент точной эмоциональной интерпретации отдельных сообщений. Более корректным является применение модели для агрегированного анализа, где отдельные ошибки частично сглаживаются при усреднении оценок по нескольким сообщениям внутри временного окна. Поэтому практический результат работы состоит не в точной диагностике эмоций, а в демонстрации приватного pipeline: локальная классификация – удаление исходного текста – хранение агрегированных статистик.

3.5. Ограничения, этика и приватность

Разработанный модуль не предназначен для психологической или медицинской диагностики. Он определяет эмоциональную окраску текста, а не реальное эмоциональное состояние человека. Пользователь может шутить, цитировать чужие слова, использовать сарказм, скрывать эмоции или писать формально, поэтому результаты календаря следует рассматривать только как вспомогательную статистику по отправленным сообщениям.

Для практического внедрения такой функции в клавиатуру необходим явный пользовательский контроль. Анализ эмоциональной окраски должен включаться только по согласию пользователя; агрегированные данные календаря должны быть доступны для удаления; исходные сообщения не должны сохраняться после классификации и не должны попадать в

логи или отчеты об ошибках. Дополнительно модуль должен отключаться в чувствительных полях ввода: паролях, одноразовых кодах, банковских реквизитах, адресах электронной почты и других формах, где эмоциональный анализ не нужен и может создавать лишний риск.

В рамках данной работы реализован исследовательский Python-прототип, а не полноценная Android- или iOS-клавиатура. Поэтому измерения задержки и потребления памяти отражают поведение моделей в экспериментальном окружении и используются для относительного сравнения. Для промышленного применения требуется отдельная проверка на мобильном устройстве, анализ энергопотребления, экспорт модели в мобильный формат и тестирование поведения в реальном интерфейсе ввода.

3.6. Выводы по главе

В третьей главе был реализован воспроизводимый Python-прототип локальной эмоциональной классификации для сценария будущей мобильной клавиатуры. Разработаны baseline-модели, единый интерфейс применения модели, бенчмарк качества и ресурсных характеристик, а также механизм агрегации сообщений в эмоциональный календарь. В протокол сравнения включены и измерены четыре кандидата. На корпусе RuGoEmotions лучшим по macro F1 оказался RuBERT-tiny2, однако для текущей версии прототипа была выбрана TF-IDF + Logistic Regression: среди легких локально запускаемых моделей она показала лучший macro F1 при приемлемых задержке, размере и потреблении памяти.

Заключение

В ходе выполнения работы была исследована задача локального определения эмоциональной окраски коротких русскоязычных сообщений в сценарии мобильной клавиатуры. Исходная проблема состояла в том, что современные интеллектуальные клавиатуры часто используют серверную обработку или синхронизацию пользовательского ввода, тогда как полностью локальные решения обычно уступают по функциональности и качеству анализа текста. В работе был рассмотрен один из возможных способов расширения функциональности приватной клавиатуры – локальный модуль оценки эмоциональной окраски сообщений и построения эмоционального календаря.

В первой части работы был проведен обзор исследований, связанных с приватной обработкой текста, мобильными клавиатурами, анализом коротких сообщений, русскоязычными корпусами тональности и методами эмоциональной классификации. Было показано, что задача локального анализа эмоций в коротких русскоязычных сообщениях находится на пересечении нескольких направлений: NLP для мобильных устройств, анализа тональности, лингвистики эмоций и защиты пользовательских данных. Отдельно рассмотрены ограничения мобильного запуска: размер модели, задержка инференса, потребление памяти и возможность работы без передачи текста на сервер.

Для экспериментальной части была сформирована шестиклассовая постановка задачи: тревожность, злость, грусть, забота, радость и нейтральное состояние. В качестве основного корпуса использован RuGoEmotions, приведенный к выбранной схеме классов и сбалансированный по категориям. Итоговый набор данных включал 7200 сообщений: 4608 сообщений в обучающей выборке, 1152 сообщения в валидационной и 1440 сообщений в тестовой. Такая схема позволила сравнить модели в воспроизводимых условиях и использовать macro F1 как основную метрику качества.

В работе были реализованы и сравнены четыре подхода: словарный baseline, TF-IDF + Logistic Regression, fastText supervised и RuBERT-tiny2. Наибольшее значение macro F1 среди рассмотренных моделей показал RuBERT-tiny2, что подтверждает преимущество контекстных трансформерных представлений для эмоциональной классификации. При этом словарный baseline оказался лучшим по размеру и скорости, но заметно уступил остальным моделям по качеству. fastText supervised показал низкую задержку по сравнению с TF-IDF+LR, однако уступил ей по macro F1. Для реализации текущего локального прототипа была выбрана TF-IDF + Logistic Regression: ее macro F1 на тестовой выборке составил 0,423, что является лучшим результатом среди легких моделей, а ресурсные характеристики остались приемлемыми для исследовательского локального модуля.

На основе выбранной модели был разработан Python-прототип, имитирующий ключевой сценарий будущей мобильной клавиатуры. Модуль принимает текст сообщения, воз-

вращает эмоциональную метку, уверенность и распределение оценок по классам. Дополнительно реализован механизм построения эмоционального календаря по CSV-диалогу. Сообщения разделяются по отправителю, дате и периоду дня, после чего для каждого временного окна усредняются выходные оценки модели. Такой подход позволяет учитывать неопределенность модели и не сводить период к простому голосованию по отдельным меткам.

Работа модуля была продемонстрирована на двух реалистичных русскоязычных диалогах за период с 01.05.26 по 05.05.26. В результате были построены четыре календаря динамики эмоциональной окраски сообщений: для Александра, Екатерины, Анастасии и мамы. Для каждого пользователя календарь содержит эмоцию, уверенность и количество сообщений по трем периодам дня: утро, день и вечер. При этом концепция модуля соответствует требованию приватности: после классификации для календаря достаточно хранить агрегированные статистики, а не исходные тексты сообщений.

Ограничением работы является то, что качество моделей оценивалось на корпусе RuGoEmotions, который не полностью совпадает с реальными мессенджерными данными конкретного пользователя. Кроме того, выходные оценки модели используются как относительные оценки уверенности и не должны интерпретироваться как точные психологические измерения эмоционального состояния. Демонстрационные диалоги позволяют проверить пользовательский сценарий эмоционального календаря, но не заменяют полноценную разметку реальных переписок экспертами.

В качестве дальнейшего развития работы целесообразно расширить набор данных за счет реальных коротких русскоязычных сообщений с ручной разметкой, провести дополнительную калибровку выходных оценок модели, оптимизировать fastText и RuBERT-tiny2 под мобильный запуск и исследовать возможность экспорта выбранного решения в мобильный формат. Отдельным направлением является разработка интерфейса эмоционального календаря и механизмов персонализации, которые сохраняют приватность и не требуют передачи исходных сообщений на сервер.

Таким образом, результат работы следует рассматривать как исследовательский и инженерный прототип локального анализа эмоциональной окраски сообщений, а не как завершенное промышленное мобильное приложение.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- [1] Hard A., Rao K., Mathews R., Ramaswamy S., Beaufays F., Augenstein S., Eichner H., Kiddon C., Ramage D. Federated Learning for Mobile Keyboard Prediction [Электронный ресурс]. – 2018. – arXiv:1811.03604. – URL: <https://arxiv.org/abs/1811.03604> (дата обращения: 11.06.2026).
- [2] Wang K., Mathews R., Kiddon C., Eichner H., Beaufays F., Ramage D. Federated Evaluation of On-device Personalization [Электронный ресурс]. – 2019. – arXiv:1910.10252. – URL: <https://arxiv.org/abs/1910.10252> (дата обращения: 11.06.2026).
- [3] Suliman M., Leith D. Two Models are Better than One: Federated Learning Is Not Private For Google GBoard Next Word Prediction [Электронный ресурс]. – 2022. – arXiv:2210.16947. – URL: <https://arxiv.org/abs/2210.16947> (дата обращения: 11.06.2026).
- [4] Baziotis C., Athanasiou N., Chronopoulou A., Kolovou A., Paraskevopoulos G., Ellinas N., Narayanan S., Potamianos A. NTUA-SLP at SemEval-2018 Task 1: Predicting Affective Content in Tweets with Deep Attentive RNNs and Transfer Learning [Электронный ресурс]. – 2018. – arXiv:1804.06658. – URL: <https://arxiv.org/abs/1804.06658> (дата обращения: 11.06.2026).
- [5] Felbo B., Mislove A., Søgaard A., Rahwan I., Lehmann S. Using millions of emoji occurrences to learn any-domain representations for detecting sentiment, emotion and sarcasm [Электронный ресурс]. – 2017. – arXiv:1708.00524. – URL: <https://arxiv.org/abs/1708.00524> (дата обращения: 11.06.2026).
- [6] Demszky D., Movshovitz-Attias D., Ko J., Cowen A., Nemade G., Ravi S. GoEmotions: A Dataset of Fine-Grained Emotions [Электронный ресурс]. – 2020. – arXiv:2005.00547. – URL: <https://arxiv.org/abs/2005.00547> (дата обращения: 11.06.2026).
- [7] Seara. RuGoEmotions: Russian adaptation of GoEmotions dataset [Электронный ресурс]. – Hugging Face Datasets. – URL: https://huggingface.co/datasets/seara/ru_go_emotions (дата обращения: 12.06.2026).
- [8] Mohammad S. M., Turney P. D. Crowdsourcing a Word-Emotion Association Lexicon [Электронный ресурс]. – 2013. – arXiv:1308.6297. – URL: <https://arxiv.org/abs/1308.6297> (дата обращения: 11.06.2026).
- [9] Salton G., Buckley C. Term-weighting approaches in automatic text retrieval // Information Processing & Management. – 1988. – Vol. 24, No. 5. – P. 513–523. – DOI: 10.1016/0306-4573(88)90021-0.

- [10] Manning C. D., Raghavan P., Schütze H. Introduction to Information Retrieval. – Cambridge: Cambridge University Press, 2008. – URL: <https://nlp.stanford.edu/IR-book/> (дата обращения: 19.06.2026).
- [11] Cox D. R. The Regression Analysis of Binary Sequences // Journal of the Royal Statistical Society. Series B. – 1958. – Vol. 20, No. 2. – P. 215–242.
- [12] Hastie T., Tibshirani R., Friedman J. The Elements of Statistical Learning: Data Mining, Inference, and Prediction. – 2nd ed. – New York: Springer, 2009. – URL: <https://hastie.su.domains/ElemStatLearn/> (дата обращения: 19.06.2026).
- [13] Joulin A., Grave E., Bojanowski P., Mikolov T. Bag of Tricks for Efficient Text Classification [Электронный ресурс]. – 2016. – arXiv:1607.01759. – URL: <https://arxiv.org/abs/1607.01759> (дата обращения: 11.06.2026).
- [14] Joulin A., Grave E., Bojanowski P., Douze M., Jégou H., Mikolov T. FastText.zip: Compressing text classification models [Электронный ресурс]. – 2016. – arXiv:1612.03651. – URL: <https://arxiv.org/abs/1612.03651> (дата обращения: 11.06.2026).
- [15] Vaswani A., Shazeer N., Parmar N., Uszkoreit J., Jones L., Gomez A. N., Kaiser L., Polosukhin I. Attention Is All You Need [Электронный ресурс]. – 2017. – arXiv:1706.03762. – URL: <https://arxiv.org/abs/1706.03762> (дата обращения: 19.06.2026).
- [16] Devlin J., Chang M.-W., Lee K., Toutanova K. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding [Электронный ресурс]. – 2019. – arXiv:1810.04805. – URL: <https://arxiv.org/abs/1810.04805> (дата обращения: 19.06.2026).
- [17] Kuratov Y., Arkhipov M. Adaptation of Deep Bidirectional Multilingual Transformers for Russian Language [Электронный ресурс]. – 2019. – arXiv:1905.07213. – URL: <https://arxiv.org/abs/1905.07213> (дата обращения: 19.06.2026).
- [18] Sanh V., Debut L., Chaumond J., Wolf T. DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter [Электронный ресурс]. – 2019. – arXiv:1910.01108. – URL: <https://arxiv.org/abs/1910.01108> (дата обращения: 11.06.2026).
- [19] Sun Z., Yu H., Song X., Liu R., Yang Y., Zhou D. MobileBERT: a Compact Task-Agnostic BERT for Resource-Limited Devices [Электронный ресурс]. – 2020. – arXiv:2004.02984. – URL: <https://arxiv.org/abs/2004.02984> (дата обращения: 11.06.2026).
- [20] Panopoulos I., Nikolaidis S., Venieris S. I., Venieris I. Exploring the Performance and Efficiency of Transformer Models for NLP on Mobile Devices [Электронный ресурс]. – 2023. – arXiv:2306.11426. – URL: <https://arxiv.org/abs/2306.11426> (дата обращения: 11.06.2026).

- [21] Rahman M. W. U., Abrar M. M., Copening H. G., Hariri S., Shao S., Satam P., Salehi S. Quantized Transformer Language Model Implementations on Edge Devices [Электронный ресурс]. – 2023. – arXiv:2310.03971. – URL: <https://arxiv.org/abs/2310.03971> (дата обращения: 11.06.2026).
- [22] Kotelnikov E. Current Landscape of the Russian Sentiment Corpora [Электронный ресурс]. – 2021. – arXiv:2106.14434. – URL: <https://arxiv.org/abs/2106.14434> (дата обращения: 11.06.2026).
- [23] Шаховский В. И. Лингвистическая теория эмоций. – М.: Гнозис, 2008.
- [24] Бабенко Л. Г. Лексические средства обозначения эмоций в русском языке. – Свердловск: Изд-во Уральского университета, 1989.
- [25] Kotelnikova A., Paschenko D., Bochenina K., Kotelnikov E. Lexicon-based Methods vs. BERT for Text Sentiment Analysis [Электронный ресурс]. – 2021. – arXiv:2111.10097. – URL: <https://arxiv.org/abs/2111.10097> (дата обращения: 13.06.2026).
- [26] Razova E., Vychezhzhanin S., Kotelnikov E. Does BERT look at sentiment lexicon? [Электронный ресурс]. – 2021. – arXiv:2111.10100. – URL: <https://arxiv.org/abs/2111.10100> (дата обращения: 13.06.2026).
- [27] Loukachevitch N., Rusnachenko N. Sentiment Frames for Attitude Extraction in Russian [Электронный ресурс]. – 2020. – arXiv:2006.10973. – URL: <https://arxiv.org/abs/2006.10973> (дата обращения: 13.06.2026).
- [28] Golubev A., Rusnachenko N., Loukachevitch N. RuSentNE-2023: Evaluating Entity-Oriented Sentiment Analysis on Russian News Texts [Электронный ресурс]. – 2023. – arXiv:2305.17679. – URL: <https://arxiv.org/abs/2305.17679> (дата обращения: 13.06.2026).
- [29] Shiffman S., Stone A. A., Hufford M. R. Ecological Momentary Assessment // Annual Review of Clinical Psychology. – 2008. – Vol. 4. – P. 1–32. – DOI: 10.1146/annurev.clinpsy.3.022806.091415. – URL: <https://doi.org/10.1146/annurev.clinpsy.3.022806.091415> (дата обращения: 13.06.2026).
- [30] Dodds P. S., Harris K. D., Kloumann I. M., Bliss C. A., Danforth C. M. Temporal Patterns of Happiness and Information in a Global Social Network: Hedonometrics and Twitter // PLOS ONE. – 2011. – Vol. 6, No. 12. – e26752. – DOI: 10.1371/journal.pone.0026752. – URL: <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0026752> (дата обращения: 13.06.2026).

- [31] Tumer K., Ghosh J. Linear and Order Statistics Combiners for Pattern Classification [Электронный ресурс]. – 1999. – arXiv:cs/9905012. – URL: <https://arxiv.org/abs/cs/9905012> (дата обращения: 13.06.2026).
- [32] Guo C., Pleiss G., Sun Y., Weinberger K. Q. On Calibration of Modern Neural Networks [Электронный ресурс]. – 2017. – arXiv:1706.04599. – URL: <https://arxiv.org/abs/1706.04599> (дата обращения: 13.06.2026).

Приложение

В приложении приведены ключевые фрагменты программной реализации: обучение легких моделей-кандидатов, включая выбранную TF-IDF + Logistic Regression и fastText supervised, применение сохраненной модели прототипа и построение эмоционального календаря по CSV-диалогу. Полный код проекта расположен в каталоге `src/emotion_keyboard`.

Листинг 1. Обучение TF-IDF+LR и fastText supervised

```
from pathlib import Path
import shutil
import tempfile

import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import FeatureUnion, Pipeline

from .labels import LABELS
from .models import SklearnEmotionModel
from .preprocess import normalize_text

ROOT = Path(__file__).resolve().parents[2]
DATA_DIR = ROOT / "data" / "processed"
MODELS_DIR = ROOT / "models"

def load_split(name: str) -> pd.DataFrame:
    frame = pd.read_csv(DATA_DIR / f"{name}.csv")
    frame["text"] = frame["text"].map(normalize_text)
    return frame

def train_tfidf_lr(train: pd.DataFrame) -> SklearnEmotionModel:
    features = FeatureUnion(
        [
            (
                "word",
```

```

        TfidfVectorizer(
            analyzer="word",
            ngram_range=(1, 2),
            min_df=2,
            sublinear_tf=True,
        ),
    ),
    (
        "char",
        TfidfVectorizer(
            analyzer="char_wb",
            ngram_range=(3, 5),
            min_df=2,
            sublinear_tf=True,
        ),
    ),
]
)
pipeline = Pipeline(
    [
        ("features", features),
        (
            "clf",
            LogisticRegression(
                max_iter=2000,
                class_weight="balanced",
                random_state=42,
            ),
        ),
    ],
)
pipeline.fit(train["text"], train["label"])
return SklearnEmotionModel(labels=LABELS, pipeline=pipeline)

```

```

def write_fasttext_input(train: pd.DataFrame, path: Path) -> None:
    with path.open("w", encoding="utf-8", newline="\n") as handle:
        for _, row in train.iterrows():
            text = str(row["text"]).replace("\n", " ")

```

```

        handle.write(f"__label__{row['label']} {text}\n")

def train_fasttext_supervised(train: pd.DataFrame, output_dir: Path) -> str:
    import fasttext

    output_dir.mkdir(parents=True, exist_ok=True)
    for stale_name in ("model.ftz", "model.bin"):
        (output_dir / stale_name).unlink(missing_ok=True)

    with tempfile.TemporaryDirectory(
        prefix="fasttext_undergrad_",
    ) as tmp_name:
        train_path = Path(tmp_name) / "train_fasttext.txt"
        write_fasttext_input(train, train_path)
        model = fasttext.train_supervised(
            input=str(train_path),
            lr=0.5,
            epoch=25,
            wordNgrams=2,
            dim=100,
            minn=3,
            maxn=6,
            loss="softmax",
            thread=4,
            verbose=0,
        )

        model_file = "model.bin"
        try:
            model.quantize(input=str(train_path), retrain=True, cutoff=100000)
            model_file = "model.ftz"
        except Exception:
            model_file = "model.bin"

        temp_model_path = Path(tmp_name) / model_file
        model.save_model(str(temp_model_path))
        shutil.copy2(temp_model_path, output_dir / model_file)
    return model_file

```

```

def main() -> None:
    train = load_split("train")
    tfidf_model = train_tfidf_lr(train)
    tfidf_model.save(MODELS_DIR / "tfidf_lr", kind="tfidf_lr")
    train_fasttext_supervised(train, MODELS_DIR / "fasttext_supervised")

if __name__ == "__main__":
    main()

```

Листинг 2. Применение сохраненной модели

```

from dataclasses import dataclass
from pathlib import Path

from .models import EmotionModel, load_model

@dataclass(frozen=True)
class Prediction:
    label: str
    confidence: float
    scores: dict[str, float]

class EmotionAnalyzer:
    def __init__(self, model: EmotionModel):
        self.model = model

    @classmethod
    def load(cls, path: str | Path) -> "EmotionAnalyzer":
        return cls(load_model(path))

    def predict(self, text: str) -> Prediction:
        scores = self.model.predict_proba([text])[0]
        label = max(scores, key=scores.get)
        return Prediction(
            label=label,

```

```

        confidence=float(scores[label]),
        scores=scores,
    )

```

Листинг 3. Обработка диалога и построение календаря

```

from dataclasses import dataclass
from pathlib import Path

import pandas as pd

from .analyzer import EmotionAnalyzer
from .labels import LABELS

DEFAULT_DATES = [
    "01.05.26",
    "02.05.26",
    "03.05.26",
    "04.05.26",
    "05.05.26",
]
PERIODS = ["утро", "день", "вечер"]

@dataclass(frozen=True)
class PeriodMood:
    sender: str
    date: str
    period: str
    dominant_emotion: str
    confidence: float
    message_count: int
    scores: dict[str, float]

def get_period(time_value: str) -> str:
    hour = int(str(time_value).split(":", 1)[0])
    if 6 <= hour < 12:
        return "утро"

```

```

    if 12 <= hour < 18:
        return "день"
    return "вечер"

def predict_dialog(
    dialog_path: Path,
    analyzer: EmotionAnalyzer,
) -> pd.DataFrame:
    dialog = pd.read_csv(dialog_path)
    texts = dialog["Текст сообщения"].fillna("").astype(str).tolist()
    class_scores = analyzer.model.predict_proba(texts)
    predictions = [max(row, key=row.get) for row in class_scores]

    result = dialog.copy()
    result["prediction"] = predictions
    result["confidence"] = [
        float(row[label]) for row, label in zip(class_scores, predictions)
    ]
    for label in LABELS:
        result[label] = [float(row.get(label, 0.0)) for row in class_scores]
    result["period"] = result["Время отправки"].apply(get_period)
    return result

def aggregate_period(
    group: pd.DataFrame,
    sender: str,
    date: str,
    period: str,
) -> PeriodMood:
    if group.empty:
        return PeriodMood(
            sender=sender,
            date=date,
            period=period,
            dominant_emotion="нет данных",
            confidence=0.0,
            message_count=0,

```

```

        scores={label: 0.0 for label in LABELS},
    )

scores = {
    label: float(group[label].mean())
    for label in LABELS
}
dominant = max(scores, key=scores.get)
return PeriodMood(
    sender=sender,
    date=date,
    period=period,
    dominant_emotion=dominant,
    confidence=float(scores[dominant]),
    message_count=int(len(group)),
    scores=scores,
)

def build_period_summary(
    predictions: pd.DataFrame,
    dates: list[str],
) -> pd.DataFrame:
    rows = []
    senders = sorted(
        predictions["Отправитель"].dropna().astype(str).unique()
    )
    for sender in senders:
        sender_rows = predictions[
            predictions["Отправитель"].astype(str) == sender
        ]
        for current_date in dates:
            date_rows = sender_rows[
                sender_rows["Дата"].astype(str) == current_date
            ]
            for period in PERIODS:
                group = date_rows[date_rows["period"] == period]
                mood = aggregate_period(
                    group,

```

```

        sender,
        current_date,
        period,
    )
    rows.append(
        {
            "sender": mood.sender,
            "date": mood.date,
            "period": mood.period,
            "dominant_emotion": mood.dominant_emotion,
            "confidence": mood.confidence,
            "message_count": mood.message_count,
            **mood.scores,
        }
    )
return pd.DataFrame(rows)

```

```

def build_calendars(
    dialog_paths: list[Path],
    output_dir: Path,
) -> None:
    output_dir.mkdir(parents=True, exist_ok=True)
    analyzer = EmotionAnalyzer.load("models/tfidf_lr")

    predicted_dialogs = []
    for path in dialog_paths:
        result = predict_dialog(path, analyzer)
        result.to_csv(
            output_dir / f"{path.stem}_predictions.csv",
            index=False,
            encoding="utf-8",
        )
        predicted_dialogs.append(result)

    all_predictions = pd.concat(predicted_dialogs, ignore_index=True)
    summary = build_period_summary(all_predictions, DEFAULT_DATES)
    summary.to_csv(
        output_dir / "calendar_summary.csv",

```



```
    index=False,  
    encoding="utf-8",  
)
```